Integration between the capture engine and the other stages

The capture engine is the first stage of the pipeline. Its only output is a normalized capture JSON that follows `docs/CONTRACT.md`. The protocol stage (protocol stage) and the UI stage (GUI stage) both consume that one file. Neither stage reads a PCAP directly.

```
PCAP/PCAPNG  --capture engine-->  normalized capture JSON  --+--> protocol stage (rules, hypotheses)
                                                             +--> UI stage (sessions, hex, provenance)
```

Handing the capture to the protocol stage

The protocol stage works from the normalized JSON, not from the capture. Produce it with the CLI:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

The stream bytes for each direction are in `sessions[].directions.<A_to_B|B_to_A>.bytes_b64`, and every byte range has a provenance entry. That is everything needed to frame messages and to point a field result back at a packet.

Handing the capture to the UI stage

The GUI stage reads the same file. The session list, endpoints, diagnostics, and provenance ranges map directly onto the session tree, the hex view, and the packet detail panel. The UI should not re-parse the capture.

Check scenario

This is the end-to-end check for the seam between the capture engine and the other two stages. Run it after any change to the capture module.

1. Generate the corpus (deterministic; safe to repeat):

```
python -m scripts.generate_corpus
```

1. Normalize the primary corpus capture:

```
mkdir -p out
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

1. Validate the output against the contracted schema:

```
python - <<'PY'
import json
from jsonschema import Draft202012Validator

schema = json.load(open("docs/schemas/capture.schema.json"))
data = json.load(open("out/corpus_01.json"))
Draft202012Validator(schema).validate(data)
print("ok:", len(data["sessions"]), "sessions")
PY
```

The primary capture has three sessions, each with both directions populated. Streams should carry the bytes of the recorded transactions, and every provenance range should resolve to a real packet index.

1. Tie a transaction to the wire. Take a line from `tests/corpus/corpus_journal.md`, for example:

```
1700000000.004 | read | temperature | 1595
```

Find the packet with that timestamp in the capture, and use `provenance.lookup` to move from a stream offset back to that packet (see `docs/CAPTURE_API.md`, Reading provenance).

1. Open the JSON in the GUI when the GUI stage is ready. The session tree should list the three sessions, the hex view should render the stream bytes with provenance, and the defects capture should show its gap and its ambiguous overlap as diagnostics.

Defects capture

`tests/corpus/corpus_capture_defects.pcapng` carries the cases the downstream stages must handle honestly:

- a retransmitted request (extra provenance, no extra bytes),
- an out-of-order request (reordered by sequence),
- a request with a missing segment (`gap` diagnostic, response direction),
- a response byte pair contradicted later (`ambiguity` diagnostic).

When the protocol stage frames this capture, the affected messages should be classified `incomplete` or `ambiguous`, never `matched`.

Boundaries

- The capture engine does not interpret bytes. It reports observations and provenance only.
- The protocol and UI stages do not read PCAP. If either needs bytes the JSON does not carry, that is a contract change and must be agreed with the participant first.

Reference export for the protocol stage

`tests/corpus/reference_export/` holds the normalized export of the three generated corpus records, committed so the protocol stage and the GUI can build their corpus smoke test and reference investigation without calling the capture module at all.

File	Sessions	Carries
`corpus_capture_01.normalized.json`	3	clean read/write/measure traffic
`corpus_capture_02.normalized.json`	2	same protocol, different values
`corpus_capture_defects.normalized.json`	1	a `gap` in `A_to_B`, an `ambiguity` in `B_to_A`

Rebuild them from the captures with:

```
bash tests/corpus/reference_export/regenerate.sh
```

The files are byte-for-byte reproducible from a given commit. `tests/integration/test_reference_export.py` fails if a committed file stops matching what the capture module produces, so the protocol stage never works against a stale export. A rule written against a stream from `corpus_capture_01` must be checked against `corpus_capture_02` (same protocol, different values) for counterexamples, and against `synthetic_live` (different byte layout) to confirm it is not accidentally right.

Project format on disk

An investigation is a directory produced by `src.project`:

```
project.madrigal/
  manifest.json          index of everything below, all paths relative
  captures/              the original captures, copied in
  logs/                  action log
  rules/                 interpretation rules, one file per version
  results/               rule application results
  annotations.sqlite     researcher annotations
  reports/               generated Markdown and HTML reports
```

`manifest.json` records `contract_version`, `project_version`, `name`, `created_at`, `updated_at`, `captures` (relative path + `sha256`), `rules` (id + version + path + `sha256`), `results`, and `annotations`. Because every path is relative and each capture carries its digest, the directory can be copied to another directory or machine and opened there unchanged:

```
from pathlib import Path
from src.project import Project

project = Project.create(Path("work/project.madrigal"), "cap1 investigation")
project.add_capture(Path("captures/cap1.pcapng"))
project.add_rule(Path("rules/set_parameter.json"), "set_parameter_request", 3)
project.export(Path("work/cap1.zip"))

# On another machine:
restored = Project.import_(Path("work/cap1.zip"), Path("work/restored"))
assert restored.verify_captures() == []  # capture digests still match
assert restored.verify_rules() == []  # rule digests still match
```

`Project.import_` checks every capture and every registered rule against its recorded digest and refuses a tampered archive, deleting the partial directory. See the project section of `docs/CONTRACT.md`.

Report seam

The protocol stage builds an investigation dictionary and the report module renders it. The dictionary keys are `hypotheses`, `counterexamples`, `rule_versions`, `scope`, and `open_questions` (see `src/report/model.py` for the exact shape). Render both forms:

```
from pathlib import Path
from src.report import render_html, render_markdown

render_markdown(investigation, Path("reports/investigation.md"))
render_html(investigation, Path("reports/investigation.html"))
```

The Markdown report is the deliverable; the HTML page is the same content styled per `docs/STYLE.md` and self-contained, so it renders without network access.